

Q1: AI terminology?

Ans: Gen AI(ChatGPT), Agentic AI(GitHub Copilot agent mode), Prompt, Token, LLM, knowledge cutoff dates

Q2: How you are using AI in development?

Ans: Coding assistance, RAG, MCP servers

Q3: What is MCP server?

Ans: Introduced by Anthropic. A connector between an AI and external data sources, files, or tool like Jira, sharepoint/wiki. There are lot of open-source MCP servers. MCP is an open-source standard so No AI Lock-in like only chatgpt or any other.

Q4: How you implement RAG?

Ans: Embedding models(Free, Commercial), Vector database(Milvus, Qdrant, PG), similarity search, LLM chat model

Q1: What docker

Ans:

Q2: What is kubernetes?

Ans:

Q3: Single/Multi region deployment

Ans: INTERNET => DNS(Route 53) => Firewall(WAF) => CDN(CloudFront) => (1) Angular(s3)
(2) Gateway -> ALB -> Microservices

UI: Route 53 → CloudFront → S3

Backend: Route 53 → WAF → ALB → Spring Cloud Gateway → MSAs

Q4: public and private subnet?

Ans: Public: ALB

Private: Gateway → Microservices → Database/Kafka/Redis

private ECS services may still need Internet access—for downloading something/deps so they use NAT Gateway

Q5: What is VPC?

Ans: Isolated private network and ip range inside this we can divide in subnets like private, public.
For making public attach Internet Gateway & Update Route Table.

Q6: Why aws vpn use?

Ans: For connecting private subnet resources like db etc. from laptop. Without VPN whitelist your laptop's public IP in the RDS Security Group.

Q7: Route 53?

Ans: 53 is a port. Working: User => Google Public DNS(8.8.8.8) => Route 53

Q8: ALB vs Elastic Load Balancing(ELB)?

Ans: ELB is a service that can have multiple type of load balancer. ALB is a one type of it.

Q9: CDN(CloudFront) cache management?

Ans: after deploying new version invalidate current cache. index.html → short cache / revalidate frequently

*.js, *.css, images → long cache

Q10: In AWS Account IAM users or roles vs Cognito

Ans: IAM → Who can access/manage AWS resources?

Cognito → Who can log in to my application? like any hosted website/portal

Identity Provider: Cognito, Keycloak, Okta

Q11: Cloudwatch?

Ans: Default retention: Never Expire

Log Group = collection of logs

Log Stream = stream/source of logs

Log Event = individual log entry

Alarm = monitors a metric and triggers an action

Logs Insights = queries CloudWatch logs

1 Task with 2 containers → potentially 2 log streams

1 Service with 10 tasks → potentially 10 log streams per container

Q12: What is serverless?

Ans: In this we don't manage infra. There are lot of services those are provided as serverless like lambda, Fargate, S3, DynamoDB

Q13: Inter container communication in ECS?

Ans: If in same task definition then localhost:8080

if in separate task/services definition then use AWS cloud map or ALB or service connect(service-name).

Q14: Lambda vs Fargate?

Ans: Lambda is Event driven and trigger based, used for short lived task. We upload code zip file.

Fargate is used for long running task and is triggered/managed by ECS. ECS add it as a service(long running) or Task(run to completion jobs).

Q15: How to run a cron job in aws?

Ans: Scheduler(EventBridge/AWS Batch) that fires at a specific time, compute target(Lambda/Fargate) that executes code.

Q1: How to break singleton pattern in java?

Ans: Reflection(Throw a RuntimeException inside the private constructor if an instance already exists),

Serialization and Deserialization(Implement the readResolve() method), Object Cloning(Override the clone() method and explicitly throw a CloneNotSupportedException), Multithreading(Use a Double-Checked Locking), Multiple ClassLoaders

The Ultimate Fix: The Enum Singleton

Q2: What is Backend for Frontend(BFF) design pattern?

Ans: Build a separate, dedicated backend service for each distinct type of UI like web app, mobile app, smart TV.

Advantages: Fewer Network Requests,

Q1: Elasticsearch(9.5.4) terminology?

Ans: Elasticsearch is a distributed search and analytics engine built on Apache Lucene. Cluster->Nodes->Index

Index, Index Alias, document(_id,_source), Field, Mapping(Fields+Data Types), Analyzer(Character/Token Filter, Tokenizer), Inverted Index, Query(Match,Term,Bool,Range)

Q2: Relational Database vs. Elasticsearch Mapping

Ans:Database/Table=Index, Row=Document, Column=Field, Schema=Mapping, Primary Key=_id

Q3: Relational Database & Elasticsearch underlying data structure?

Ans: Elasticsearch(Inverted Index), RDBMS(B-tree, B+ tree)

Q4: Types of index in es?

Ans:Regular index, Data stream(time-series data), Searchable Snapshot (Frozen Index), Hidden index, Index alias

Q5: Required things for index creation?

Ans: Nothing is mandatory except index name. Important:number_of_shards, number_of_replicas, mappings(can cause performance issues if missed)

Q6: stop-word filter(comes under token filters) with an analyzer

Ans: the, is, on, the

Input Text -> Tokenizer -> Token Filters(Lowercase,stop-word) -> Final Tokens

Q7: Relevance scoring in Elasticsearch: _score

Ans: BM25 as the default similarity algorithm. BM25 considers factors like: Term frequency, Inverse document frequency, Field length

Q8: Query format

Ans:

```
{
  "query": {
    "match": {
      "title": {
        "query": "kids shoes",
        "operator": "and",
        "minimum_should_match": "75%"
      }
    }
  }
}
```

Q9: Response format

Ans:

```
{
  "took": 5,
  "_shards": {
  },
  {
    "hits": {
      "hits": [
```

```

{
  "_score": 2.81,
  "_source": {
    "title": "Kids shoes"
  }
},
{
  "_score": 1.42,
  "_source": {
    "title": "Shoes for kids"
  }
}
]
}
}
}

```

Q10: Elasticsearch Deployment, distribution & scaling

Ans: Cluster, Node(servers), Shards, replicas

Q11: What are primary shards and replica shards?

Ans: Shards are two types: primaries and replicas. Sharding is the process of dividing large dataset into smaller more manageable parts called shards.

No. of primary shards in an index is fixed at time of index creation , but No. of replica shards can be changed at any time.

When we index a document, Elasticsearch determines which shard to store it in based on a sharding algorithm which is typically using the document's ID or routing value

$\text{shard_num} = \text{hash}(\text{_routing}/\text{_id}) \% \text{num_primary_shards}$

Q12:Elasticsearch endpoints? GET, POST, PUT, DELETE

Ans: POST `/_{index}/_doc/{id:optional}` => POST `/_{index}/_update/{id}`

GET `/_{index}/_doc/{id}`

DELETE `/_{index}/_doc/{id}`

GET `/_{index}/_search` => GET `/_search` == search in all index

GET `/_cat/indices?v`

GET `/_{index}/_mapping` Or GET `/_{index}/_settings`

Q13: What is `_bulk` API(for mass imports) and `_reindex`?

Ans:`_reindex` API (Internal Data Copying):copy documents from one index to another

Q14: Pagination — `from/size` vs `search_after`

Ans: size, from, `max_result_window`: 10,000 Document Cap, `search_after`

Q15: Elasticsearch security

Ans: enable in this file: `elasticsearch.yml`

Q16: Searching: exact match vs full text search?

Ans:match: full text search, other overloaded option like for word sequence:match_phrase
term or terms: exact match

range: Date, Numeric

Q17: How to sort data in response?

Ans: By default sorts results by their relevance score(_score). Dates, Numbers, Keywords, Booleans: No configuration needed.
Text/Sentences, Arrays, Nested Structures requires configuration.

Q18: Aggregations(grouping), and scripting.

Ans: like count of each product sale

Q19: Internal working of storing data?

Ans: When a document is indexed, Elasticsearch sends it to Lucene. Lucene creates an inverted index.

Our created index: it contains 2 parts:(1)inverted index (2)_source=original JSON document

Q1: Java threads are dependent on OS. If something critical goes wrong at the OS level, it will immediately impact your Java threads. So what to do for this in Java?

Ans: Use Virtual Threads (Java 21)

Q2: LTS versions

Ans: Java 8, Java 11, Java 17, Java 21, Java 25

Q3: Java 17 features

Ans: Sealed Classes, Records, Pattern Matching for instanceof & switch, Text Blocks (multi-line strings)

Some Performance and Security improvement, obsolete feature Removals.

Q4: Java 21 features

Ans: Virtual Threads, Sequenced Collections, Record Patterns

Q5: ExecutorService (interface) vs ForkJoinPool (Class implementing ExecutorService)

Ans: ExecutorService: (1) One shared queue for all threads (2) Algorithm: FIFO/LIFO task pulling from central queue (3) ParallelStream/Sort, Virtual thread

ForkJoinPool: (1) One queue per worker thread (2) Algorithm: Work-stealing:

Q6: Future (Java 5) vs CompletableFuture (Java 8)

Ans: Future: Blocking

CompletableFuture: Non-blocking, Chaining, Combining, exceptionally

Q0: MSA design pattern?

Ans: Eureka, Circuit breaker, api gateway, CQRS, CAP, 12 Principal

Q1: Service registry/discovery system?

Ans: Eureka(Alternatives:AWS Cloud Map/Service Connect, Consul, K8S DNS). Eureka mostly is used in VM-based deployment. Eureka is primarily a service registry/discovery system. If we have multiple instance of service then returns a list of ip array.

Q2: Distributed transaction?

Ans: Saga

Q3: Why we use API Gateway?

Ans: Like Spring Cloud Gateway, Kong. It mainly handles cross-cutting concerns.

- 1) Single entry point
- 2) Routing
- 3) Authentication and authorization: token validation
- 4) Rate limiting: 100 req/sec
- 5) SSL/TLS termination: configuring HTTPS certificates
- 6) Request/response transformation

Q4: What is API Gateway vs Reverse Proxy?

Ans: API gateway is a specialized type of reverse proxy with extra capabilities.

Q5: Can API Gateway call multiple microservices?

Ans: Yes, Gateway can aggregate responses. if aggregation becomes complex, Use a BFF (Backend for Frontend).

Q6: How do you make API Gateway highly available?

Ans: Don't run only one Gateway instance. AWS Route 53(multi region) => ALB(single region)
=>Gateway instances => microservices

Q7: How to send spring boot app log to cloudwatch?

Ans: Configure the ECS task definition with the awslogs log driver: it uses default console logs. Those logs go to stdout and ECS sends them to CloudWatch.

Application running directly on EC2: EC2 → log file(/var/log/application.log) → CloudWatch Agent → CloudWatch

Q8: How to connect to s3 in boot application?

Ans: AWS SDK for Java, AWS access key and secret key(used while we set up aws cli), DefaultCredentialsProvider(<user>\.aws\credentials)

Local development: Configure AWS credentials using the AWS CLI:aws configure ==>C:\Users\<your-user>\.aws\credentials

Production: Attach IAM role(Temporary AWS credentials) to the ECS task

Q9: Config server? Alternative AWS Secret manager

Ans: Spring Cloud Config Server or Consul, Config Client, version control using Git

Q10: Circuit breaker? like Resilience4j or Hystrix

Ans: To stopping cascading failures when services becomes slow or stops working
We return fallback method response and support retry as well.

Q11: Service Discovery and Service Registry(centralized DB) in Microservices:

Ans: It is 2 types

(1)Client-Side Service Discovery: Netflix Eureka, Consul

(2)Server-Side Service Discovery: NGINX, AWS ELB

Q12: Distributed transaction Operation possibilities i.e resources involved

Ans: Database, Async api call(External API), Broker, Filesystem, Cache

1)DB & DB = Saga (2)DB & broker = Transactional Outbox (3) DB & Async api call (4)DB & Filesystem (5)DB + Cache(No need of transaction) (6)Broker + Broker (7)API + API

Outbox: Outbox Event, Outbox table, outbox processor, marks message as processed or delete

Q0: Types of Token

Ans: JWT, Opaque,

Q1: What is JWT(JSON Web Token)?

Ans: JWT is composed of 3 parts separated by dots (.): Header, Payload, and Signature. JWT use Base64URL encoding. JWT Header & Payload are encoded but Signature are both signed and encoded.

Note: Signature is created from the encoded header + encoded payload, then the resulting signature is Base64URL-encoded

Q2: Why we use Authorization: Bearer <JWT>

Ans: OAuth 2.0 defines the Bearer Token usage for HTTP authorization

Q3: Do we use JWT in both Authentication & Authorization

Ans: Login(username/password or clientId/Secret) → authenticate user → issue JWT

Authentication: is established by validating the JWT and identifying the subject.

Authorization: JWT contains claims such as roles or permissions, scopes

Q4: Standard JWT claims those are mandatory?

Ans: There are no mandatory claims in the JWT. Header(alg, typ, kid), Payload(iss, sub, aud, exp, iat, jti, roles, scope, custom claims), Signature

Q5: What is OAuth 2.0

Ans: OAuth2 is an authorization framework. Allows client application to access protected resources on behalf of a user or itself, without sharing the user's credentials with the client. Main components are Resource Owner(user), Client, Authorization Server, and Resource Server.

Ex: Login to LinkedIn with Google => Resource Owner(Me), Client(LinkedIn), Authorization Server(Google), & Resource Server(Google APIs)

Q6: Authorization Server & Identity Provider?

Ans: Keycloak, Auth0, Okta, AWS Cognito(for end-users), AWS IAM Identity Center(employee & developer access), Azure AD

Q7: Spring Security

Ans: Security Filter Chain=>Extract & Validate JWT => Authentication => Authorization

HTTP Request -> DelegatingFilterProxy -> Spring Security FilterChain ->

BearerTokenAuthenticationFilter -> JWT Authentication Provider -> JWT Decoder -> Validate signature + claims -> Authentication created -> SecurityContext -> Authorization filters/interceptors -> Controller

HTTP Request -> Spring Security FilterChain -> Authentication Provider -> UserDetailsService -> UserDB =>if success: SecurityContext -> SecurityContextHolder -> Authorization

-
1. Explain the Spring Security Filter Chain
 2. What is SecurityFilterChain?
 3. How do you configure Spring Security in Spring Boot 3?
 4. UserDetails vs UserDetailsService
 5. How does PasswordEncoder work?
 6. Why is BCrypt used for password hashing?
 7. How does JWT authentication work?
 8. Access Token vs Refresh Token

9. How do you secure REST APIs?
10. What is CSRF?
11. What is CORS?
12. hasRole() vs hasAuthority()
13. What is method-level security?
14. How does @PreAuthorize work?
15. What are Authentication and SecurityContext?
16. How does session management work?
17. OAuth2 vs OpenID Connect
18. How would you secure a microservices architecture using JWT/OAuth2?

Additional topics to prepare:

- * Custom authentication
- * JWT filters
- * Spring Security 6 changes

Q1: Where is used @Order In Spring Boot?

Ans: To control the execution/priority order. commonly used with filters. @Order(1) highest priority.

Request



AuthFilter



LoggingFilter



Controller

Q2: Where is used @ConditionalOn?

Ans: Spring will normally create all bean every time the application starts. But you don't want it when It isn't being used.

@Conditional allows Spring to register a bean or configuration only when a specified condition is satisfied.

Q0: @Transactional annotation's parameters?

Ans: propagation, isolation, readOnly(optimizes performance), timeout, and rollbackFor

Q1: @Transactional annotation's Propagation?

Ans: Used for defining transaction boundary behavior when a transactional method is called inside another. Default is Propagation.REQUIRED others are SUPPORTS, NOT_SUPPORTED, MANDATORY, NEVER, NESTED.

Q2: @Transactional annotation's Isolation?

Ans: It defines how safely and independently concurrent operations interact with a database. Default is Isolation.DEFAULT=db's default isolation.

Other options: READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE

Q0: All records

Ans: SELECT e.*, d.* FROM departments d INNER JOIN employees e ON e.dept_id = d.dept_id;

Q1: How do you find duplicate records like name, salaries, degrees?

Ans: SELECT emp_name, COUNT(*) AS count FROM employees GROUP BY emp_name
HAVING COUNT(*) > 1;

SELECT degree, COUNT(*) AS count FROM employees GROUP BY degree HAVING
COUNT(*) > 1;

Using Window function:

SELECT * FROM (
SELECT emp_id, emp_name, email, salary, ROW_NUMBER() OVER (PARTITION BY
salary ORDER BY emp_id) AS rowNumber FROM employees
) mandatoryAliasNameGroupedSalary WHERE rowNumber > 1;

Q2: How do you find the second-highest salary?

Ans: SELECT DISTINCT salary AS SecondHighestSalary FROM employees ORDER BY salary
DESC LIMIT 1 OFFSET 1;

Using window function:

WITH RankedSalaries AS (SELECT emp_name, salary, DENSE_RANK() OVER (ORDER BY
salary DESC) as salary_rank FROM employees)
SELECT emp_name, salary FROM RankedSalaries WHERE salary_rank = 2;

Q3: Find employees who earn more than their manager.

Ans: SELECT e.emp_name, e.salary, e.city AS employee_city, m.emp_name AS Manager, m.salary,
d.dept_name, d.location AS Dpt_Location FROM employees e INNER JOIN employees m ON
e.manager_id = m.emp_id INNER JOIN departments d ON e.dept_id = d.dept_id where e.salary >
m.salary;

Q4: Find the number of employees in each department

Ans: SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) AS employee_count FROM
departments d INNER JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_id,
d.dept_name;

Q5: Find departments having more than 2 employees

Ans: SELECT d.dept_id, d.dept_name, COUNT(e.emp_id) AS employee_count FROM
departments d INNER JOIN employees e ON d.dept_id = e.dept_id GROUP BY d.dept_id,
d.dept_name HAVING COUNT(e.emp_id) > 2;

Q6: Find the highest-paid employee in each department

Ans: SELECT d.dept_name, e.emp_name, e.salary FROM employees e JOIN departments d ON
e.dept_id = d.dept_id WHERE e.salary = (SELECT MAX(e2.salary) FROM employees e2 WHERE
e2.dept_id = e.dept_id);

Using Window function:

SELECT * FROM (SELECT emp_id, name, department, salary, ROW_NUMBER() OVER
(PARTITION BY department ORDER BY salary DESC) AS rn FROM employees) e WHERE
rn=1;

Q7: Find employees earning more than the average salary of their department

Ans: SELECT e.emp_id, e.emp_name, e.salary, d.dept_name FROM employees e JOIN
departments d ON e.dept_id = d.dept_id WHERE e.salary > (SELECT AVG(e2.salary) FROM
employees e2 WHERE e2.dept_id = e.dept_id);

Q8: Find employees along with their manager name

Ans: SELECT e.emp_id, e.emp_name, e.city AS employee_city, m.emp_name AS Manager, d.dept_name, d.location AS Dpt_Location FROM employees e INNER JOIN employees m ON e.manager_id = m.emp_id INNER JOIN departments d ON e.dept_id = d.dept_id;

Q9: Find the employees having greater than avg employees salary

Ans: SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

Q10: Find departments max salary

Ans: SELECT dept_id, MAX(salary) AS max_salary FROM employees GROUP BY dept_id;

Q11: Rank employees by salary in each department

Ans: SELECT emp_id, emp_name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num FROM employees;

Q12: RANK() vs DENSE_RANK() and what is ROW_NUMBER()?

Ans: Check gap after the tie.

Rank(gap after tie):(1)90000 (2)80000 (2)80000 (4)60000

DENSE_RANK(No gap) :(1)90000 (2)80000 (2)80000 (3)60000

SELECT emp_name, salary, RANK()/DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk FROM employees;

ROW_NUMBER(): a unique, sequential integer to each row starting at 1, regardless of duplicate values

SELECT emp_name, dept_id, salary, ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num FROM employees;

Q13: Before window functions what we used to?

Ans: correlated subqueries, self-joins